Cloud Computing

By: Ahmad Shata

Syllabus

What is Cloud Computing?

History and Cloud Evolution

Virtualization vs Containerization

Cloud Models

Application Architectures

What is Cloud Computing?

On-premise Vs Cloud

- location
- Control
- Security
- Compliance
- Cost
- Time To Market (TTM)

Cloud computing is the on-demand delivery of IT resources over the Internet with pay-as-you-go pricing. Instead of buying, owning, and maintaining physical data centers and servers, you can access technology services, such as computing power, storage, and databases, on an as-needed basis from a cloud service provider.

What is Cloud Computing?

On-premise Vs Cloud

Key Differences of On-Premise vs. Cloud

Deployment (Location)

On Premises: In an on-premises environment, resources are deployed in-house and within an enterprise’s IT infrastructure. An enterprise is responsible for maintaining the solution and all its related processes.

Cloud: In a public cloud computing environment, resources are hosted on the premises of the service provider, but enterprises are able to access those resources and use as much as they want at any given time.

What is Cloud Computing?

On-premise Vs Cloud

Key Differences of On-Premise vs. Cloud

Control

On Premises: In an on-premises environment, enterprises retain all their data and are fully in control of what happens to it, for better or worse. Companies in highly regulated industries with extra privacy concerns are more likely to hesitate to leap into the cloud before others because of this reason.

Cloud: In a cloud computing environment, the question of ownership of data is one that many companies – and vendors for that matter, have struggled with. Data and encryption keys reside within your third-party provider, so if the unexpected happens and there is downtime, you

What is Cloud Computing?

On-premise Vs Cloud

Key Differences of On-Premise vs. Cloud

Security

On Premises: Companies that have extra sensitive information, such as government and banking industries must have a certain level of security and privacy that an on-premises environment provides. Despite the promise of the cloud, security is the primary concern for many industries, so an on-premises environment, despite some of its drawbacks and price tag, make more sense.

Cloud: Security concerns remain the number one barrier to cloud computing deployment. There have been many publicized cloud breaches, and IT departments around the world are concerned. From

What is Cloud Computing?

On-premise Vs Cloud

Key Differences of On-Premise vs. Cloud

Compliance

On Premises: Many companies operate under some form of regulatory control, regardless of the industry. Perhaps the most common one is the Health Insurance Portability and Accountability Act (HIPAA) for private health information, and other government and industry regulations. For companies that are subject to such regulations, they must remain compliant and know where their data is all the time.

Cloud: Enterprises that do choose a cloud computing model must ensure that their third-party provider is up to code and in fact compliant with all of the different regulatory mandates within their industry. Sensitive data must be secured, and customers, partners, and

Expense	On-Premise	Cloud
Upfront Cost	Requires significant CapEx to get the hardware and infrastructure.	Typically operates on a subscription-based pricing model. It requires less upfront investment.
Maintenance Cost	Requires continuous maintenance resources. It includes space, power, and expert staff.	The service provider maintains the software. It reduces the need for internal maintenance resources.
Scalability Cost	Additional hardware and setup may be necessary for growth, leading to extra costs.	Cloud is scalable with the ability to adapt quickly. Changing business needs without significant additional costs.
Upgrade Cost	Upgrades can be costly as they may require new hardware or system re-configurations.	Software updates are typically included in the subscription cost. These are performed automatically by the provider.
Data Loss Cost	Potential for permanent data loss in case of system	More robust data protection

What is Cloud Computing?

On-premise Vs Cloud

Key Differences of On-Premise vs. Cloud

Time To Market (TTM)

On Premises:

- Procurement Delays
- Setup and Configuration
- Geographic Limitations
- Upfront Payment

Cloud:

- No Hardware Procurement Delays
- Pre-configured Services
- Global Reach
- Pay-as-You-Go Model

Advantages

Faster time to market

You can spin up new instances or retire them in seconds, allowing developers to accelerate development with quick deployments. Cloud computing supports new innovations by making it easy to test new ideas and design new applications without hardware limitations or slow procurement processes.

Scalability and flexibility

Cloud computing gives your business more flexibility. You can quickly scale resources and storage up to meet business demands without having to invest in physical infrastructure.

Companies don’t need to pay for or build the infrastructure needed to support their highest load levels. Likewise, they can quickly scale down if resources

Advantages

Cost savings

Whatever cloud service model you choose, you only pay for the resources you actually use. This helps you avoid overbuilding and overprovisioning your data center and gives your IT teams back valuable time to focus on more strategic work.

Better collaboration

Cloud storage enables you to make data available anywhere you are, anytime you need it. Instead of being tied to a location or specific device, people can access data from anywhere in the world from any device—as long as they have an internet connection.

Advantages

Advanced security

Despite popular perceptions, cloud computing can actually strengthen your security posture because of the depth and breadth of security features, automatic maintenance, and centralized management.

Reputable cloud providers also hire top security experts and employ the most advanced solutions, providing more robust protection.

Data loss prevention

Cloud providers offer backup and disaster recovery features. Storing data in the cloud rather than locally can help prevent data loss in the event of an emergency, such as hardware malfunction, malicious threats, or even simple user error.

Disadvantages

Vendor Reliability and Downtime

Because of technological difficulties, maintenance needs, or even cyberattacks, cloud service providers can face outages or downtime. Users may not be able to access their data or applications during these times, which can interfere with business operations and productivity.

Internet Dependency

A dependable and fast internet connection is essential for cloud computing. Business operations may be delayed or interrupted if there are connectivity problems or interruptions in the internet service that affect access to cloud services and data.

Disadvantages

Limited Control and Customization

Using standardized services and platforms offered by the cloud service provider is a common part of cloud computing. As a result, organizations may have less ability to customize and control their infrastructure, applications, and security measures. It may be difficult for some organizations to modify cloud services to precisely match their needs if they have special requirements or compliance requirements.

Data Security and Concerns about Privacy

Concerns about data security and privacy arise when sensitive data is stored on the cloud. Businesses must have faith in the cloud service provider's security procedures, data encryption, access controls, and regulatory compliance. Unauthorized access to data or data breaches can

Disadvantages

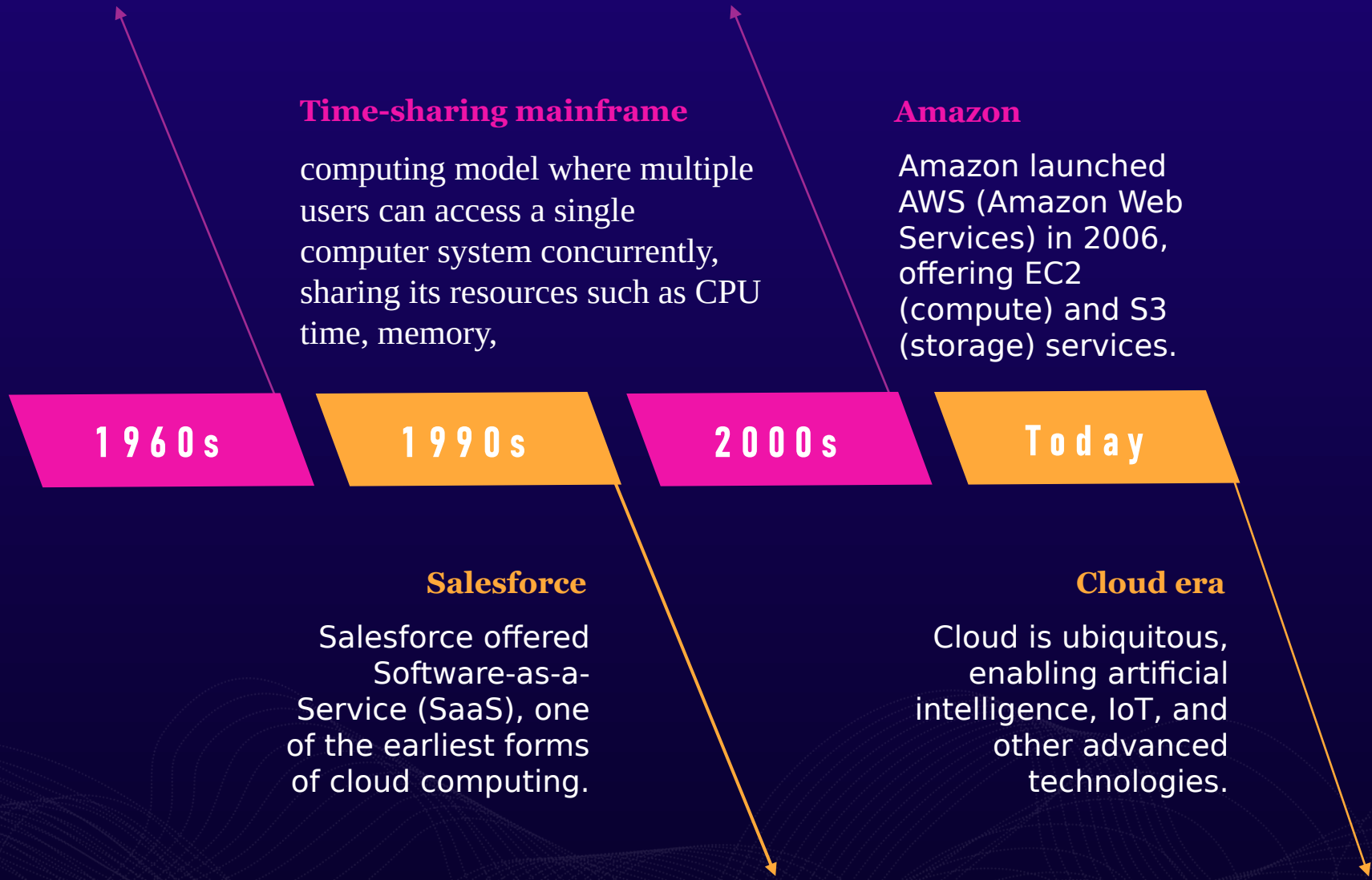
Hidden Costs and Pricing Models

Although pay-as-you-go models and lower upfront costs make cloud computing more affordable, businesses should be wary of hidden charges. Data transfer fees, additional storage costs, fees for specialized support or technical assistance, and expenses related to regulatory compliance are a few examples.

Dependency on Service Provider

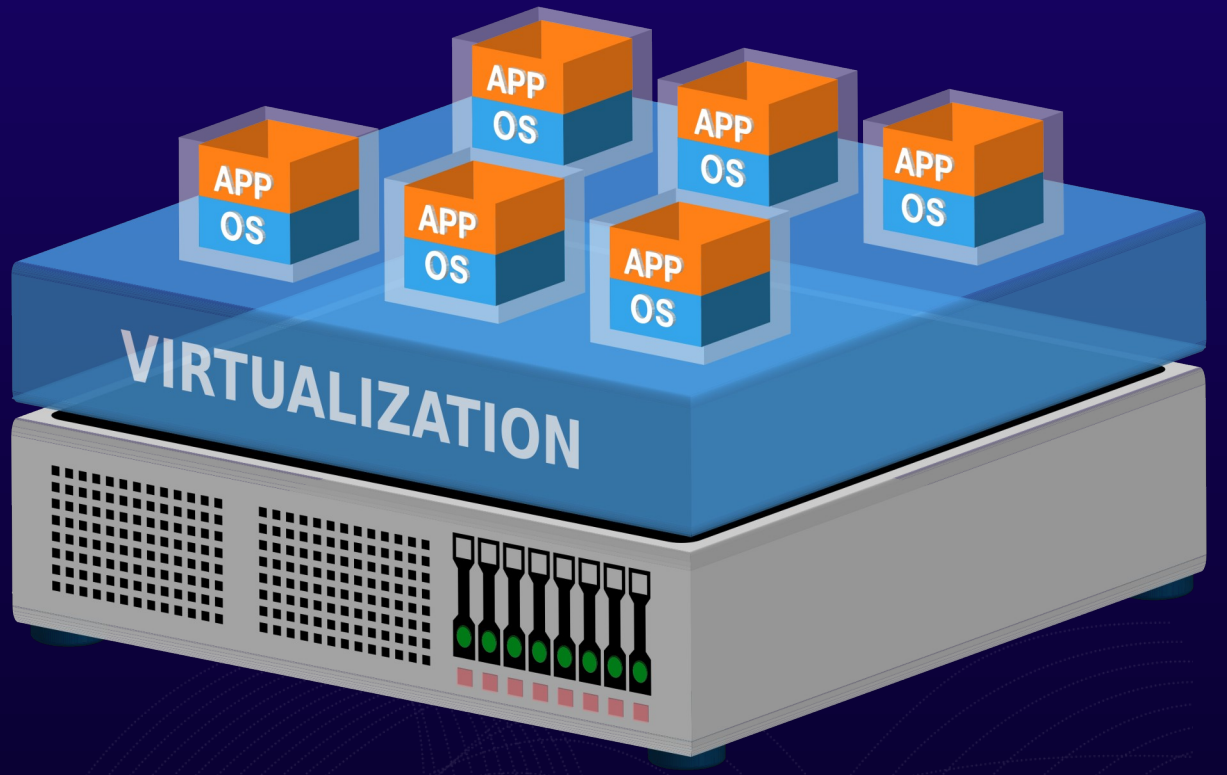
When an organization depends on a cloud service provider, it is dependent on that provider's dependability, financial security, and longevity. Users may have disruptions and difficulties switching to alternate options if the provider runs into financial difficulties, changes their pricing policy, or even closes down their services.

History


1960s
Time-sharing mainframe
computing model where multiple users can access a single computer system concurrently, sharing its resources such as CPU time, memory,
1990s
Salesforce
Salesforce offered Software-as-a-Service (SaaS), one of the earliest forms of cloud computing.
2000s
Amazon
Amazon launched AWS (Amazon Web Services) in 2006, offering EC2 (compute) and S3 (storage) services.
Today
Cloud era
Cloud is ubiquitous, enabling artificial intelligence, IoT, and other advanced technologies.

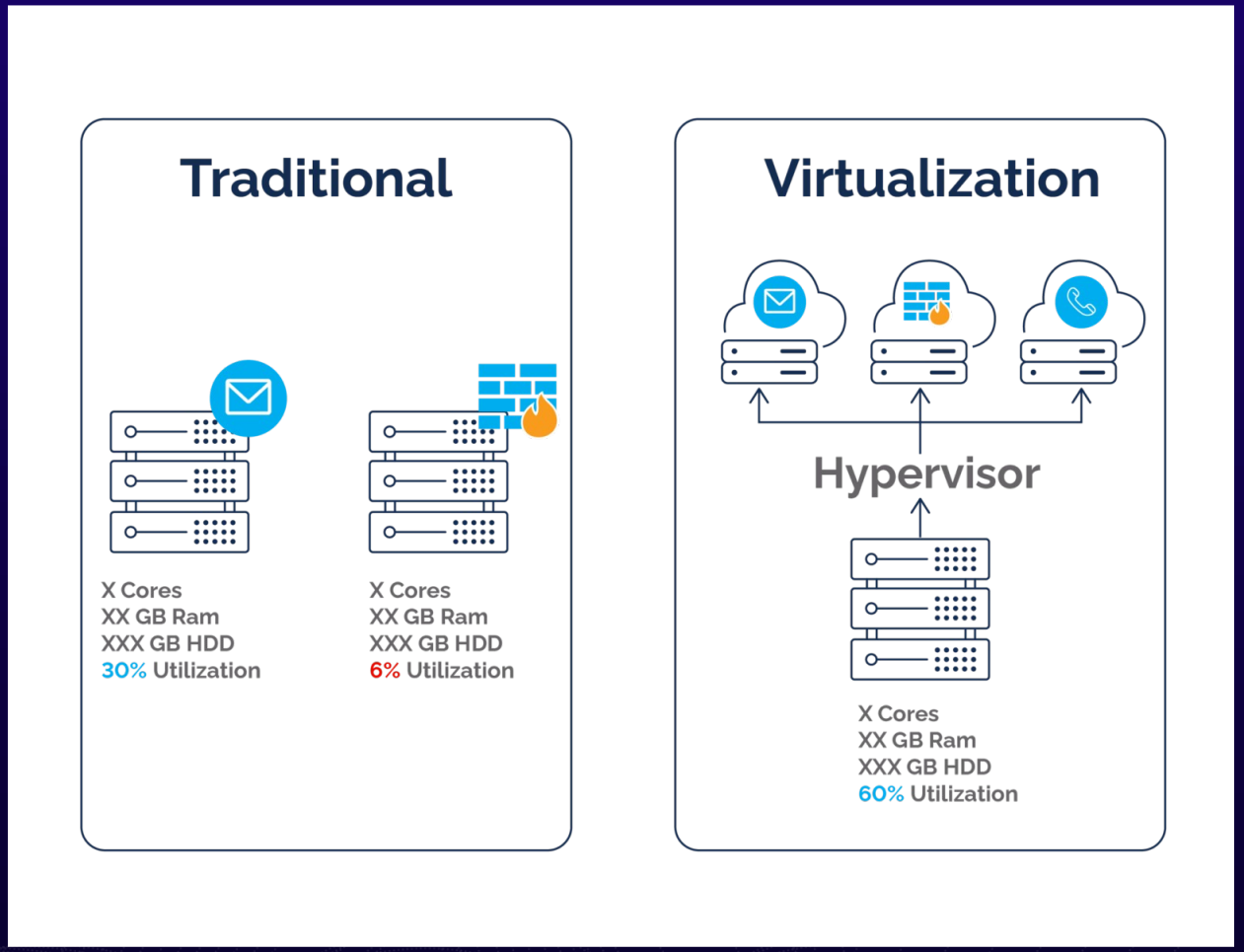
Virtualization

Virtualization is used to create an abstraction layer over computer hardware, enabling the division of a single computer's hardware components—such as processors, memory and storage—into multiple virtual machines (VMs). Each VM runs its own operating system (OS) and behaves like an independent computer, even though it is running on just a portion of the actual underlying computer hardware.



Virtualization

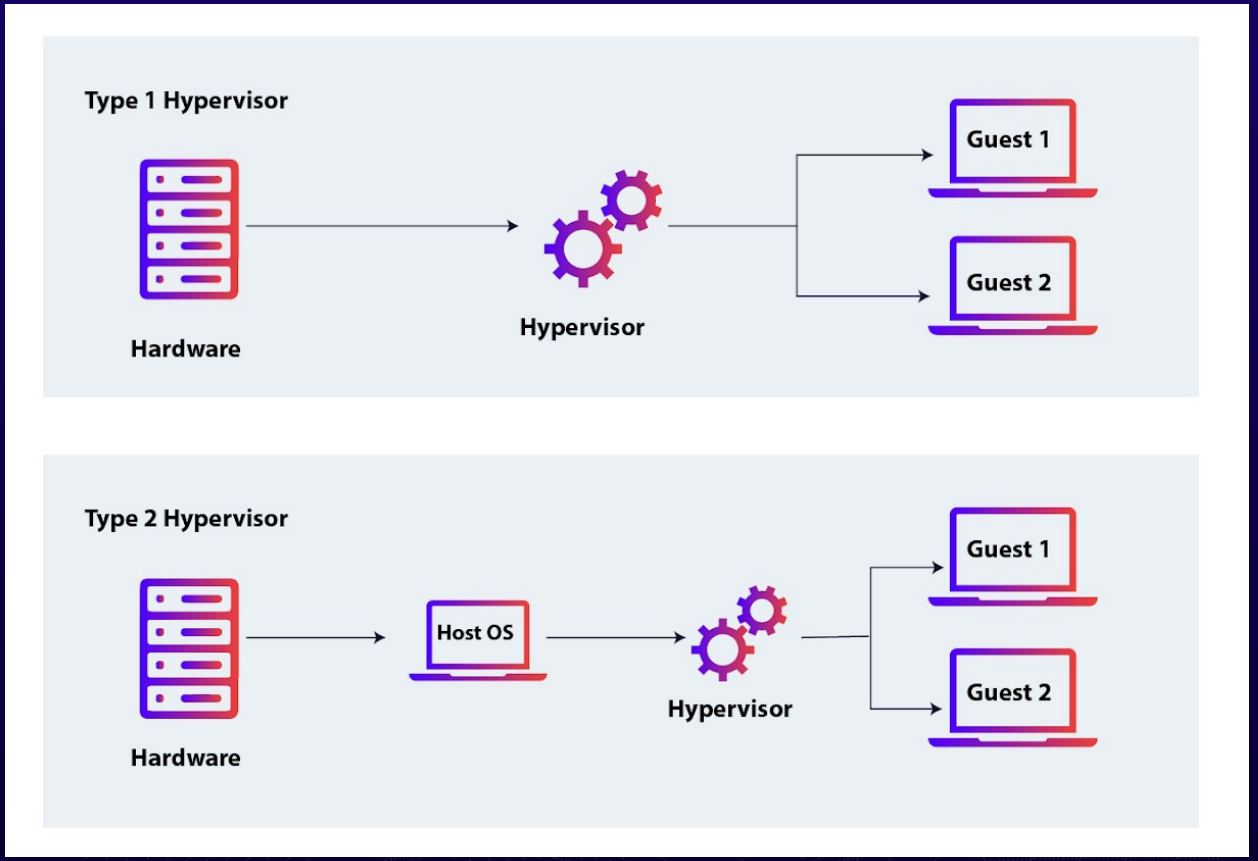
Virtualization is used to create an abstraction layer over computer hardware, enabling the division of a single computer's hardware components—such as processors, memory and storage—into multiple virtual machines (VMs). Each VM runs its own operating system (OS) and behaves like an independent computer, even though it is running on just a portion of the actual underlying computer hardware.



Virtualization

Hypervisor

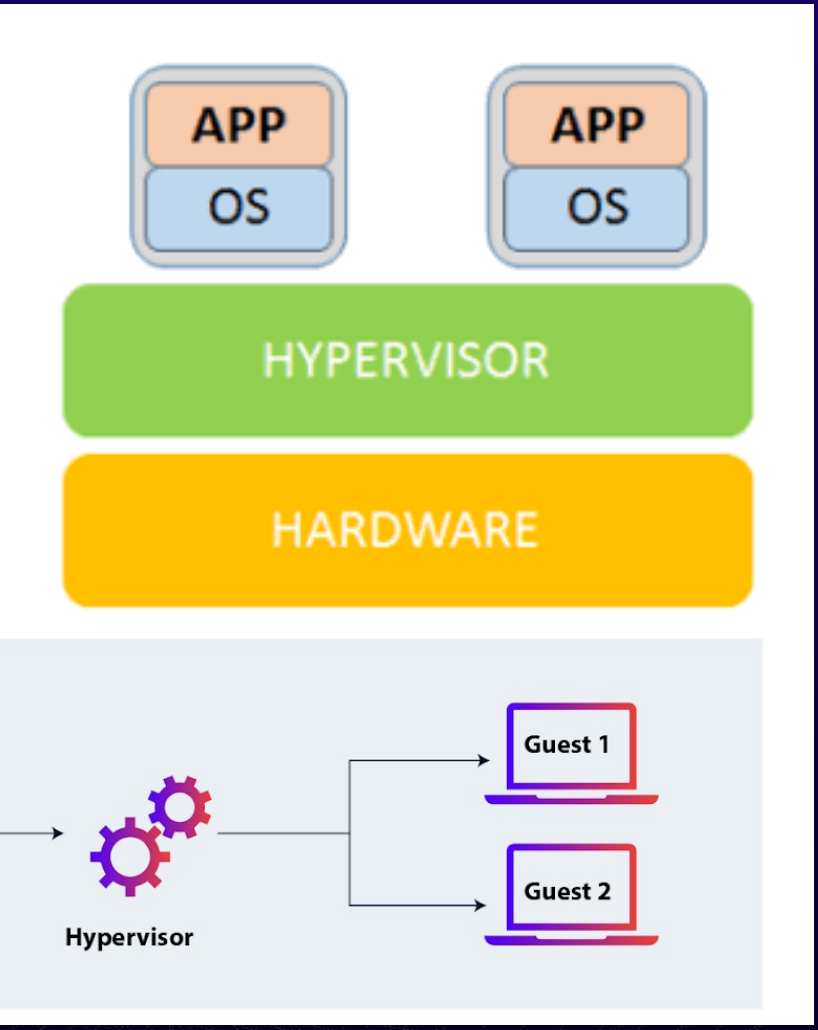
A hypervisor is a type of software or hardware used to create virtual machines and then run those virtual machines day to day. You'll sometimes see the same technology referred to as a 'virtual machine monitor,' or VMM, which is a reasonable encapsulation of what a hypervisor does.



Hypervisor

Type 1 Hypervisor

Most common in enterprise data centers, a type 1 hypervisor replaces the host's operating system and lies right on top of the hardware. For this reason, type 1 hypervisors are also called bare metal hypervisors or embedded hypervisors.



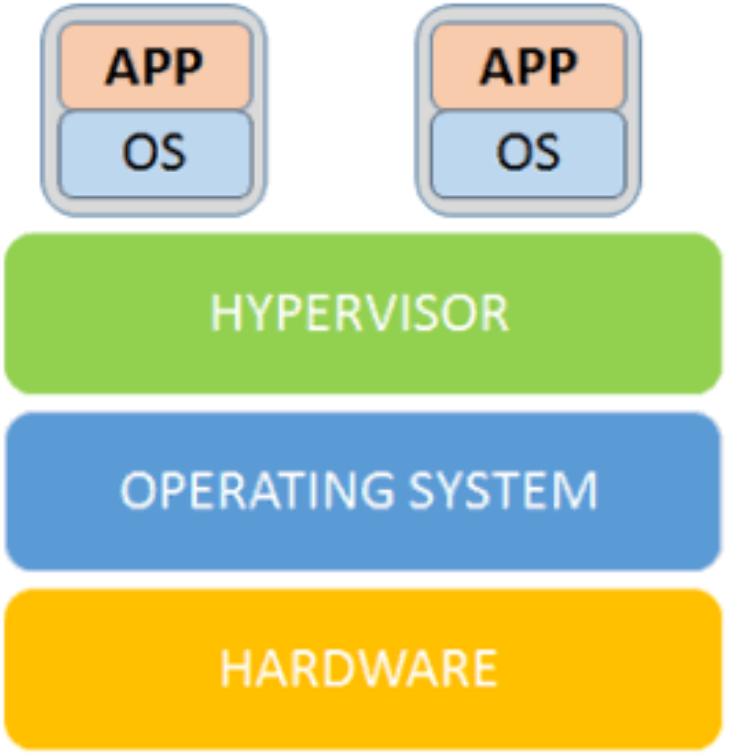
Type 1 Hypervisor Examples

- **VMware hypervisors like vSphere, ESXi**
- **Microsoft Hyper-V**
- **Oracle VM Server**
- **Citrix Hypervisor**

Hypervisor

Type 2 Hypervisors

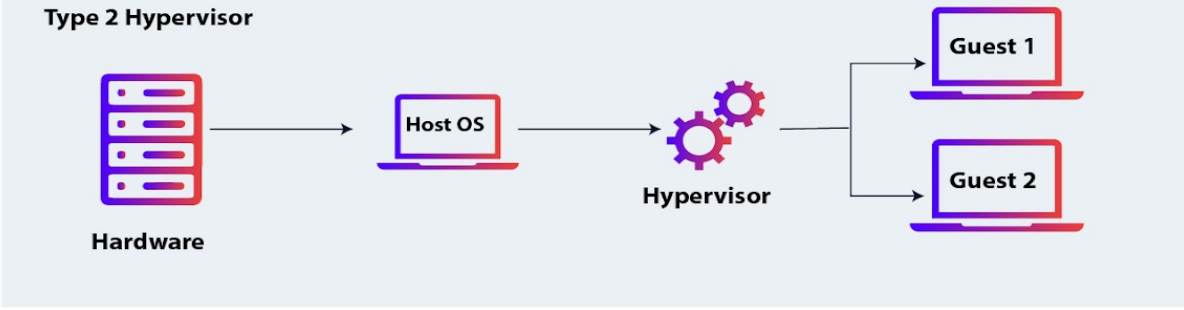
A type 2 hypervisor is hosted, running as software on the O/S, which in turn runs on the physical hardware. This form of hypervisor is typically used to run multiple operating systems on one personal computer, such as to enable the user to boot into either Windows or Linux.



Type 2 Hypervisor Examples:

VMware Workstation

Oracle VirtualBox



Virtualization

What's an example of virtualization?

Here's a common virtualization scenario: a business has three physical servers that each have a specific purpose:

One supports web traffic,
One supports company email,
One supports internal business applications.
With each physical server only being used for its dedicated purpose, the business is probably only using one-third of each server's computing capacity—even though it pays 100% of the server's maintenance costs.

With virtualization, you could split one of the servers into two virtual machines and cut your maintenance costs by 33%. This means one server could handle email and web traffic, another could host all business applications, and the third could be retired to save costs or repurposed for some other IT service.

Containerization

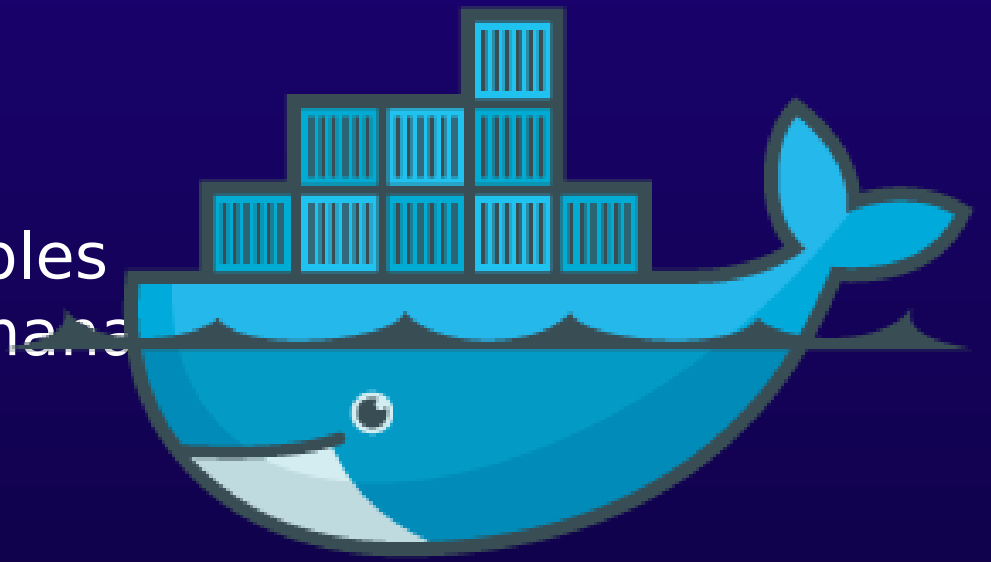
Containerization is "OS-level virtualization". It doesn't simulate the entire physical machine. It just simulates the OS of your machine.

- Multiple applications can share the same OS kernel.
- Most common container technology is "Docker".
- Containerization is the process of bundling the application code along with the libraries, configuration files, and dependencies required for the application to run cross-platform.
- It is an application-packaging approach where the code is written once and capable of executing anywhere.
- It provides less isolation than virtualizations as the containers share the same kernel unlike virtualization where each virtual machine is completely isolated from the other machines.

Containerization

Docker

Docker is an open-source framework that enables developers to build, deploy, run, update and ma... containers.



Docker utilizes LXC (Linux Containers) which refers to capabilities of the Linux kernel (specifically namespaces and control groups) which allow sandboxing processes from one another and controlling their resource allocations.
On top of this low-level foundation of kernel features, Docker offers a high-level tool with several powerful functionalities.

Namespace gives the isolation for the container with the underline host.
Cgroups gives the ability to allocate resources to the containers.

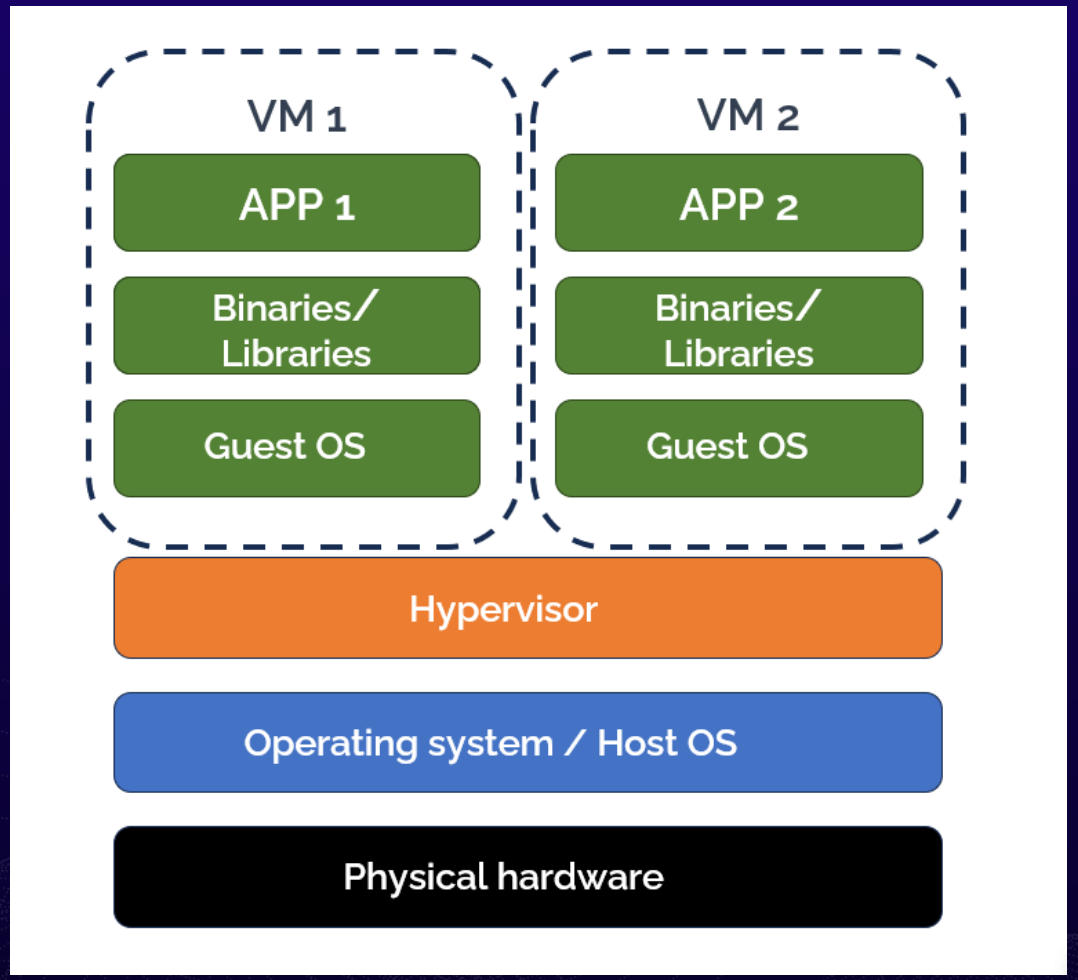
1. Docker Engine: The core component of Docker that enables container management. It includes the Docker daemon (dockerd), which runs as a background service, and the Docker CLI (docker), which provides a command-line interface for interacting with Docker.

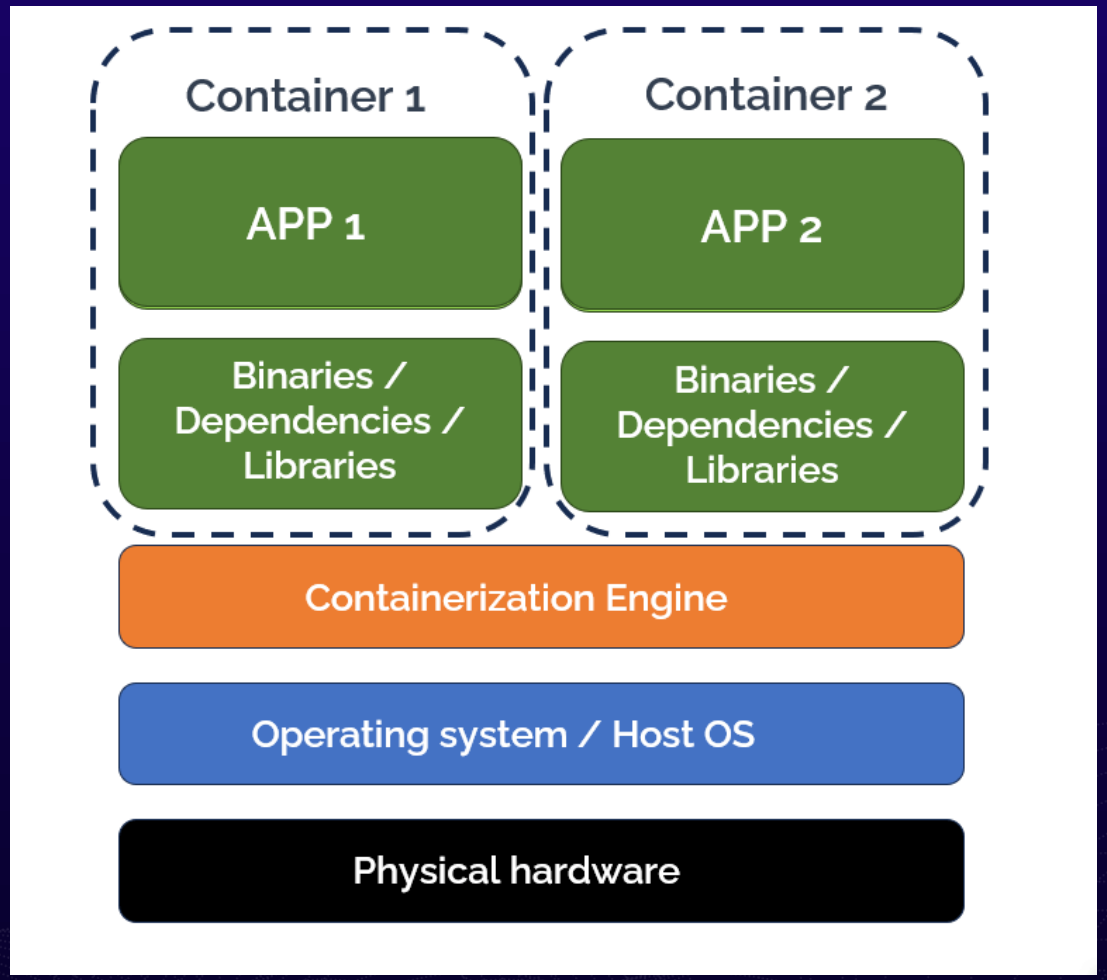
2. Docker Image: A read-only template that contains the application code, libraries, and dependencies required to run a container. Images serve as the building blocks for containers. Docker images can be based on other images, and they are versioned to allow for reproducibility.

3. Docker Container: A running instance of a Docker image. Containers are isolated from the host system and other containers, but they share the host OS kernel. This isolation ensures that applications can run consistently regardless of the underlying infrastructure.

4. Docker Registry: A repository that stores Docker images. Docker Hub is the default public registry that allows developers to access and share images.

Virtualization vs Containerization





Virtualization vs Containerization

1. Resource Overhead

When comparing containerization vs virtualization in terms of resource overhead, containerization is the clear winner. Because containers share the host system’s operating system, and do not need to run a full operating system, they are significantly more lightweight and consume fewer resources. Virtual machines, on the other hand, each require their own OS, which increases the overhead, especially when many VMs are running on the same host system.

2. Startup Time

In general, containers start up more quickly than VMs, because they don’t have to start up an entire operating system. Virtual machines take much longer to boot up. This means containers are more flexible and can be torn down and restarted whenever needed, supporting immutability, which means that a resource never changes after being deployed.

Virtualization vs Containerization

3. Portability

Both containers and virtual machines offer a high degree of portability. However, containers have a slight edge because they package the application and all of its dependencies together into a single unit, which can be run on any system that supports the container platform. Virtual machines, while also portable, are more dependent on the underlying hardware.

4. Security Isolation

In terms of security isolation, virtual machines have the advantage. Because each VM is completely isolated from the host system and other VMs, a security breach in one VM typically does not affect the others (although it is possible to compromise the hypervisor and take control of all VMs on the device). Containers, while isolated from each other, still share the host system's OS, so a breach in one container could possibly leak to

Virtualization vs Containerization

5. Scalability and Management

The lightweight nature and rapid startup time offered by containers make them ideal for scaling applications quickly and efficiently. They also lend themselves well to the microservices architecture, which can simplify the management of complex applications. Virtual machines, while also scalable, are more resource-intensive and take longer to start, making them less suitable for microservices and distributed applications.

Application Architectures

A monolithic architecture is a single, unified application where all the components and functionalities are tightly integrated into one cohesive unit. It runs as a single process, and all its parts are interdependent.

Characteristics:

Single Codebase: All components, such as the user interface (UI), business logic, and data access layers, are part of one codebase.
Tightly Coupled: Any change to one part of the system may require redeploying the entire application.
Centralized Database: Often relies on a single database for all operations.

Advantages:

Simplicity: Easier to develop and test initially due to its single codebase.
Performance: Can be faster since components communicate directly within the application.
Deployment: Requires only one deployment process for the entire application.

Challenges:

Scalability: Difficult to scale individual components independently.
Maintenance: Becomes cumbersome as the application grows in size and complexity.
Downtime: Any changes require redeploying the whole system, leading to potential downtime.

Example:

A traditional e-commerce application where the catalog, user authentication, order processing,

3-Tier Architecture

3-Tier architecture is a layered approach that divides the application into three distinct layers:

Presentation Layer: The user interface that interacts with end-users.

Business Logic Layer: Contains the rules and operations of the application (calculations, etc.).

Data Layer: Handles data storage, retrieval, and management.

Characteristics:

Separation of Concerns: Each layer is responsible for a specific function, making it modular.

Advantages:

Modularity: Easy to maintain and update individual layers without affecting others.

Scalability: Layers can be scaled independently, especially the data and business logic layers.

Flexibility: Supports integration with other systems via APIs or middleware..

Challenges:

Complexity: Slightly more complex than monolithic architectures to design and implement.

Latency: Communication between layers can introduce latency.

Deployment: Requires managing multiple components

Microservices Architecture

Microservices architecture breaks down an application into a collection of small, independent, and loosely coupled services. Each service is designed to handle a specific business capability and can be developed, deployed, and scaled independently.

Characteristics:

Decoupled Services: Each service runs in its own process and communicates via lightweight APIs.
Polyglot Programming: Different services can use different programming languages, databases, or frameworks.
Independent Deployment: Changes to one service don’t require redeploying the entire application.

Advantages:

Scalability: Individual services can be scaled as needed.
Resilience: Failure in one service doesn’t affect the entire system.
Faster Development: Teams can work on services independently, speeding up development cycles.

Challenges:

Complexity: Managing many services can be challenging, especially communication and coordination.
Latency: Services communicate over a network, which may introduce latency.
Monitoring: Requires robust monitoring and logging for troubleshooting.

Example:

A modern e-commerce platform where:
The **user service** handles authentication and user profiles.
The **catalog service** manages product listings.

Deployment Models

A cloud deployment model essentially defines where the infrastructure for your deployment resides and determines who has ownership and control over that infrastructure. It also determines the cloud’s nature and purpose.

Types of Cloud Deployment Models

- **Public Cloud**
- **Private Cloud**
- **Hybrid Cloud**
- **Multi-Cloud**

Deployment Models

Public Cloud Model

Public cloud is a commonly adopted cloud model, where **the cloud services provider owns the infrastructure and openly provides access to it for the public to consume**.
As the service provider owns the hardware and supporting networking infrastructure, it is under the service provider's full control. The service provider is responsible for the physical security, maintenance, and management of the data center where the infrastructure resides. The underlying infrastructure is, therefore, outside of the customer's control and also away from the customer's physical location.
Examples: Microsoft Azure, Amazon AWS, Google Cloud, Oracle Cloud.

Advantages of the Public Cloud Model

Low initial capital cost (Move from Capex to Opex)
High Flexibility
High (almost unlimited) scalability
High Reliability
Low maintenance costs

Disadvantages of the Public Cloud Model

Deployment Models

Private Cloud Model

A private cloud can be thought of as **an environment that is fully owned and managed by a single tenant**. This option is usually chosen to alleviate any data security concerns that might exist with the public cloud offering. Any strict cloud governance requirements can also be more easily adhered to, and the private cloud can be more easily customized.
Full control of the hardware can lead to higher performance. A customer will typically run a private cloud within their own building (on-premises) or purchase rack space in a data center in which to host their infrastructure.
However, the responsibility to manage the infrastructure also falls to the customer, creating a need for more staff with wider skills and increasing costs.
Examples: On-premises data centers using OpenStack or VMware.

Advantages of the Private Cloud Model

Increased security and control
Dedicated hardware may improve performance
High flexibility

Disadvantages of the Private Cloud Model

High cost

Deployment Models

Hybrid Cloud Model

The hybrid model **combines both public and private cloud deployment models** giving a single cloud infrastructure that is aimed at increasing flexibility and deployment options for the business. For example, applications with strict governance and data security requirements may be hosted in the business private cloud, whereas applications without these concerns, which need to be scaled on demand, could be hosted in the public cloud.
Examples: Using a private cloud for sensitive data and a public cloud for other operations.

Advantages of the Hybrid Cloud
Improved scalability
High control
Highly scalable
High fault tolerance
Cost-effective
Disadvantages of the Hybrid Cloud
Setup challenges
High management overhead

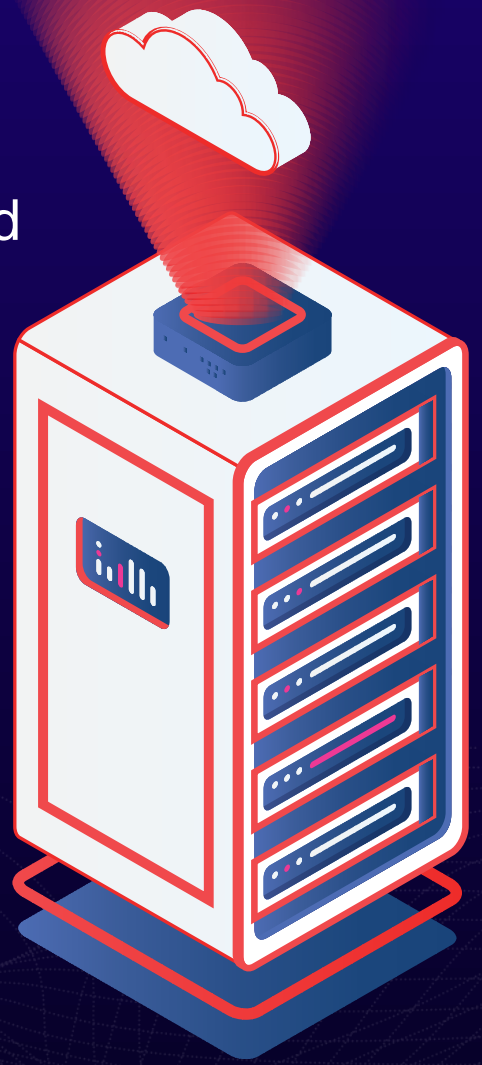
Deployment Models

Multi-Cloud Model

nearly 90% of companies are now considered multi-cloud, meaning they combine cloud services from at least two different cloud service providers, whether public or private. Adopting a multi-cloud approach gives you greater flexibility to choose the solutions that best suit your specific business needs, and also reduces the risk of vendor lock-in.

Service Models

Infrastructure as a Service (IaaS)



What It Offers:
Virtualized computing resources like virtual machines (VMs), storage, and networks.
Users control the operating system and applications.

Features:
Full control over the operating system and runtime.
Ideal for businesses needing customizable environments.
Requires technical expertise to manage.

Examples:
AWS EC2 (Elastic Compute Cloud)
Google Compute Engine
Azure Virtual Machines

Use Cases:
Hosting a website or application with custom configurations.
Disaster recovery and backup solutions

Service Models

Platform as a Service (PaaS)

What It Offers:
A complete development and deployment environment, abstracting infrastructure complexities.
Provides tools, libraries, and frameworks for developers..

Features:
Developers focus only on coding and deploying applications.
Middleware and runtime are preconfigured.
Lower operational overhead compared to IaaS.

Examples:
AWS Elastic Beanstalk
Google App Engine
Microsoft Azure App Service.

Use Cases:
Developing mobile apps quickly.
Building e-commerce platforms

Service Models

Software as a Service (SaaS)

What It Offers:
Ready-to-use software applications accessible over the internet.
No installation or management of underlying infrastructure.

Features:
Fully managed by the service provider.
Accessible through web browsers or lightweight client software.
Subscription-based pricing models.

Examples:
Google Workspace (formerly G Suite): Gmail, Google Docs, Sheets, etc.
Microsoft Office 365: Cloud-based Word, Excel, PowerPoint.

Use Cases:
Collaborative work (e.g., Google Docs).
Communication (e.g., Zoom, Slack).

Service Models

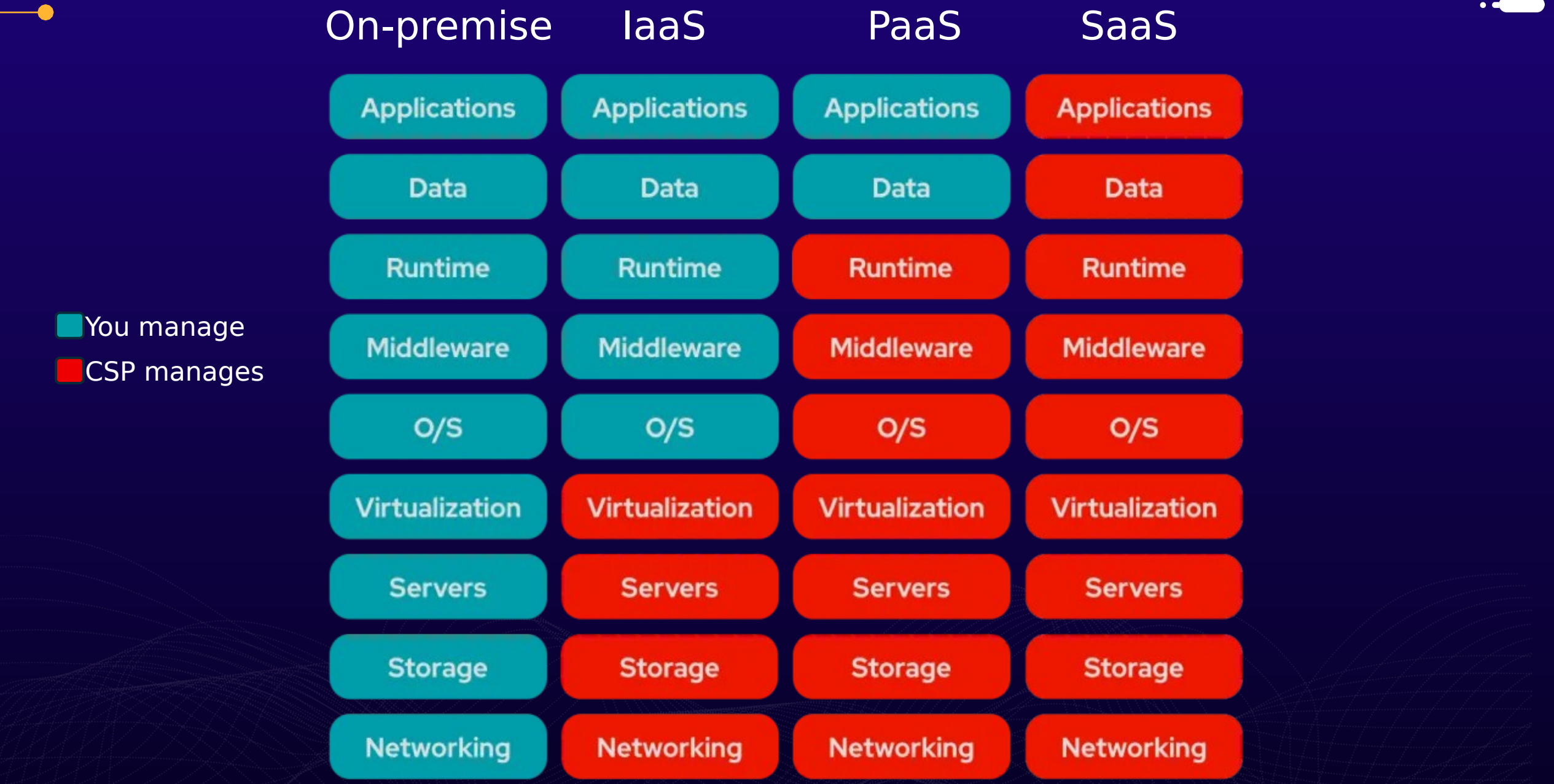
A simple analogy to help remember the difference between IaaS, PaaS, SaaS, and serverless is to think of the models like eating a cake. You could make your own from scratch (on-premises data center), where you buy all the basic ingredients to make everything like the flour and milk.
However, most of us generally don’t have enough time or don’t want to spend so much time and effort to eat a cake.

Instead, you might choose from the following options instead:

IaaS: Buying pre-packed ingredients like fresh milk and flour made by someone else that you use to cook at home.

PaaS: Order takeout or delivery where your cake is prepared for you and you don’t have to worry about the ingredients or how you’ll bake it, but you have to worry about the final look of the cake in terms of garnishing and customizing the final look,

SaaS: Call ahead to the bakery and order the exact cake you want. They prepare everything ahead of time for you so that all you have to do is show up and eat.


On-premise
IaaS
PaaS
SaaS
Applications
Data
Runtime
Middleware
O/S
Virtualization
Servers
Storage
Networking
You manage
CSP manages

Middleware

Middleware is software that acts as a bridge between different applications, systems, or components. It enables communication, data exchange, and functionality between separate or incompatible systems.

Role of Middleware

Middleware typically sits between the operating system and the application. It provides services such as messaging, authentication, and database access that are not provided by the operating system itself.

Examples of Middleware

1.Database Middleware:

1. Connects applications to databases.
2. Example: JDBC (Java Database Connectivity) for Java applications to interact with databases like MySQL or PostgreSQL.

2.Message-Oriented Middleware (MOM):

1. Enables asynchronous communication between distributed systems.
2. Example: Apache Kafka, RabbitMQ.

3.API Gateways:

1. Manage API requests and responses in a scalable and secure manner.
2. Example: AWS API Gateway.

4.Web Servers:

1. Act as middleware between client-side requests (via browsers) and the backend.

Cloud Service Providers (CSPs)

Google Cloud Platform

Microsoft
Azure

Alibaba Cloud

AWS

The AWS Cloud spans 108 Availability Zones within 34 geographic regions, with announced plans for 18 more Availability Zones and six more AWS Regions in Mexico, New Zealand, the Kingdom of Saudi Arabia, Thailand, Taiwan, and the AWS European Sovereign Cloud.

AWS Infrastructure

AWS Regions

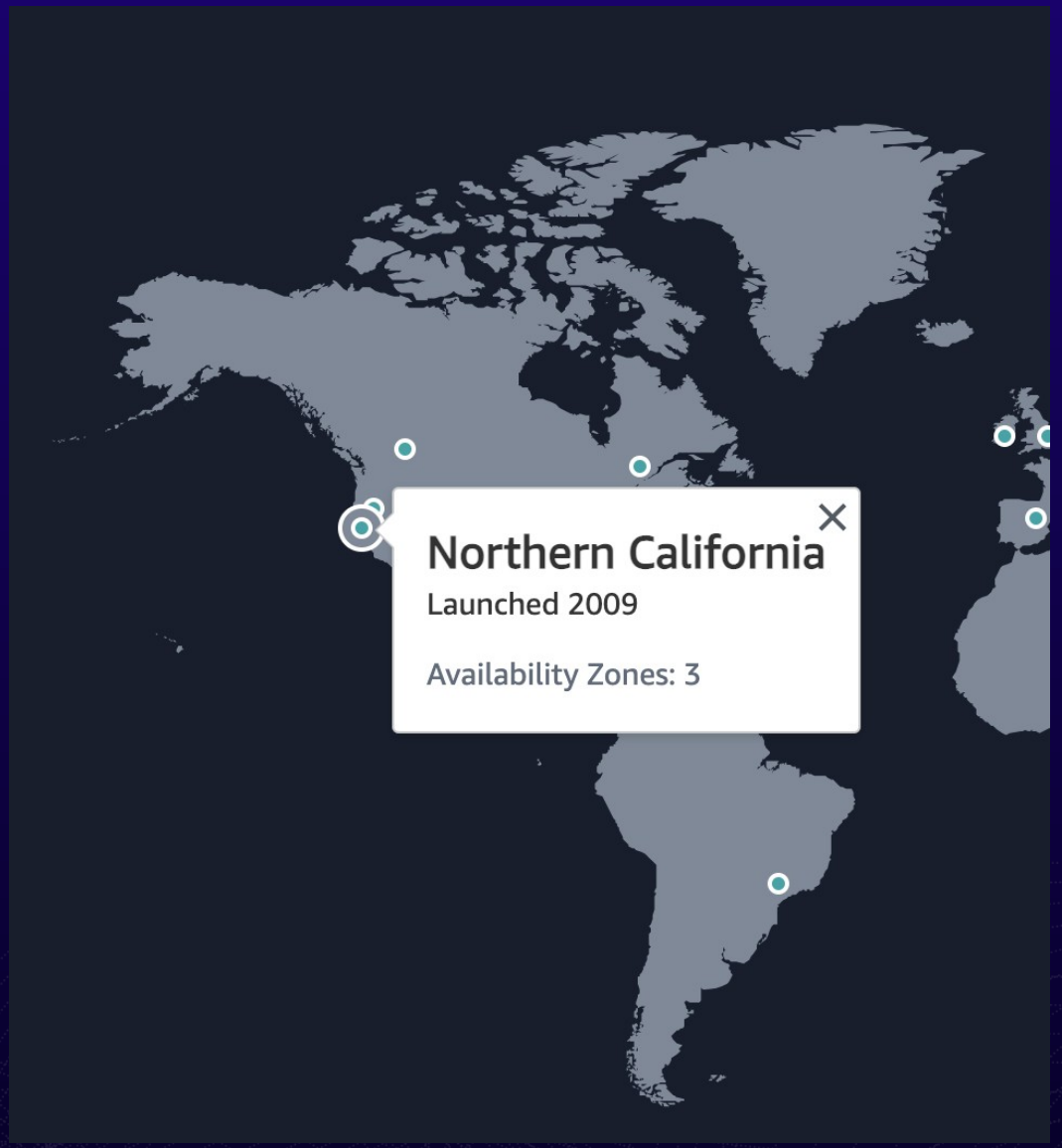
A region is a geographically isolated location where AWS has data centers. Each region is independent and consists of multiple Availability Zones.

Why Regions Exist:

- **Proximity to Customers**: Regions allow you to run applications closer to your users, reducing latency.
- **Compliance**: Some regions meet specific legal or regulatory requirements for data residency.
- **Disaster Recovery**: By spreading workloads across regions, businesses can recover faster in case of a region-wide outage.

Examples of Regions:

- **us-east-1** (North Virginia, USA)
- **eu-west-1** (Ireland)
- **ap-southeast-1** (Singapore)



AWS Infrastructure

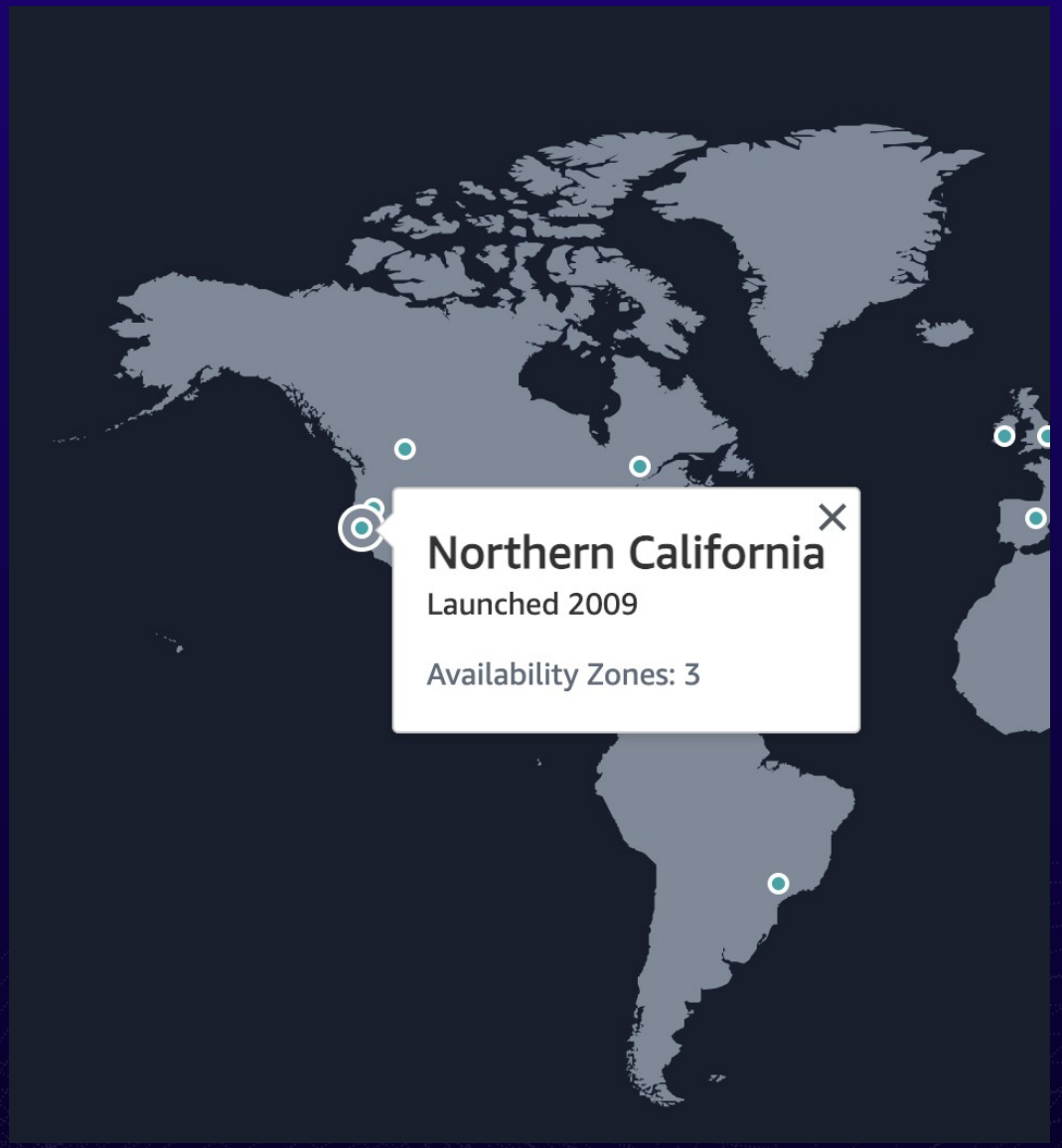
Availability Zones (AZs)
An Availability Zone is a distinct physical location within a region. Each AZ has its own power, cooling, and networking to ensure independence. Each region has at least 2 AZs

Why Multiple AZs Exist:

- **High Availability**: Distributing workloads across multiple AZs ensures that even if one AZ fails (e.g., due to a natural disaster), others remain operational.
- **Fault Tolerance**: AZs are isolated from failures in other AZs but are connected through low-latency links for high-speed communication.

Key Properties of AZs:

- At least **two AZs** exist in every AWS region.
- They are connected using high-speed, fiber-optic networking.
- Examples: A region like **us-east-1** may have



AWS Key concepts

High Availability (HA)

Fault Tolerance (FT)

Disaster Recovery (DR)

AWS Key concepts

High Availability (HA)

High Availability refers to a system's ability to remain operational and accessible for a very high percentage of time. It aims to minimize downtime, ensuring services are continuously available to users.
Measured as a percentage of uptime (e.g., **99.9% availability** translates to ~8.76 hours of downtime annually).

Fault Tolerance (FT)

Disaster Recovery (DR)

AWS Key concepts

High Availability (HA)

High Availability refers to a system's ability to remain operational and accessible for a very high percentage of time. It aims to minimize downtime, ensuring services are continuously available to users.
Measured as a percentage of uptime (e.g., **99.9% availability** translates to ~8.76 hours of downtime annually).

Fault Tolerance (FT)

Fault Tolerance ensures that a system continues to operate **without interruption** even if one or more components fail. This involves building systems that can withstand failures **seamlessly**.

Disaster Recovery (DR)

AWS Key concepts

High Availability (HA)

High Availability refers to a system's ability to remain operational and accessible for a very high percentage of time. It aims to minimize downtime, ensuring services are continuously available to users.
Measured as a percentage of uptime (e.g., **99.9% availability** translates to ~8.76 hours of downtime annually).

Fault Tolerance (FT)

Fault Tolerance ensures that a system continues to operate **without interruption** even if one or more components fail. This involves building systems that can withstand failures **seamlessly**.

Disaster Recovery (DR)

Disaster Recovery refers to the process and infrastructure set up to **restore operations after a catastrophic event**, such as data center failures, natural disasters, or cybersecurity incidents.

AWS Key Services

entity and Access Management (IAM)

- **Purpose**: Manage access to AWS services/resources securely.
- **Users**: Individual accounts representing people or apps.
- **Groups**: Collection of users sharing permissions.
- **Roles**: Temporary access for AWS services or federated users.
- **Policies**: JSON documents defining permissions (allow/deny actions).
- **Best Practices**:
  - Enable MFA for users.

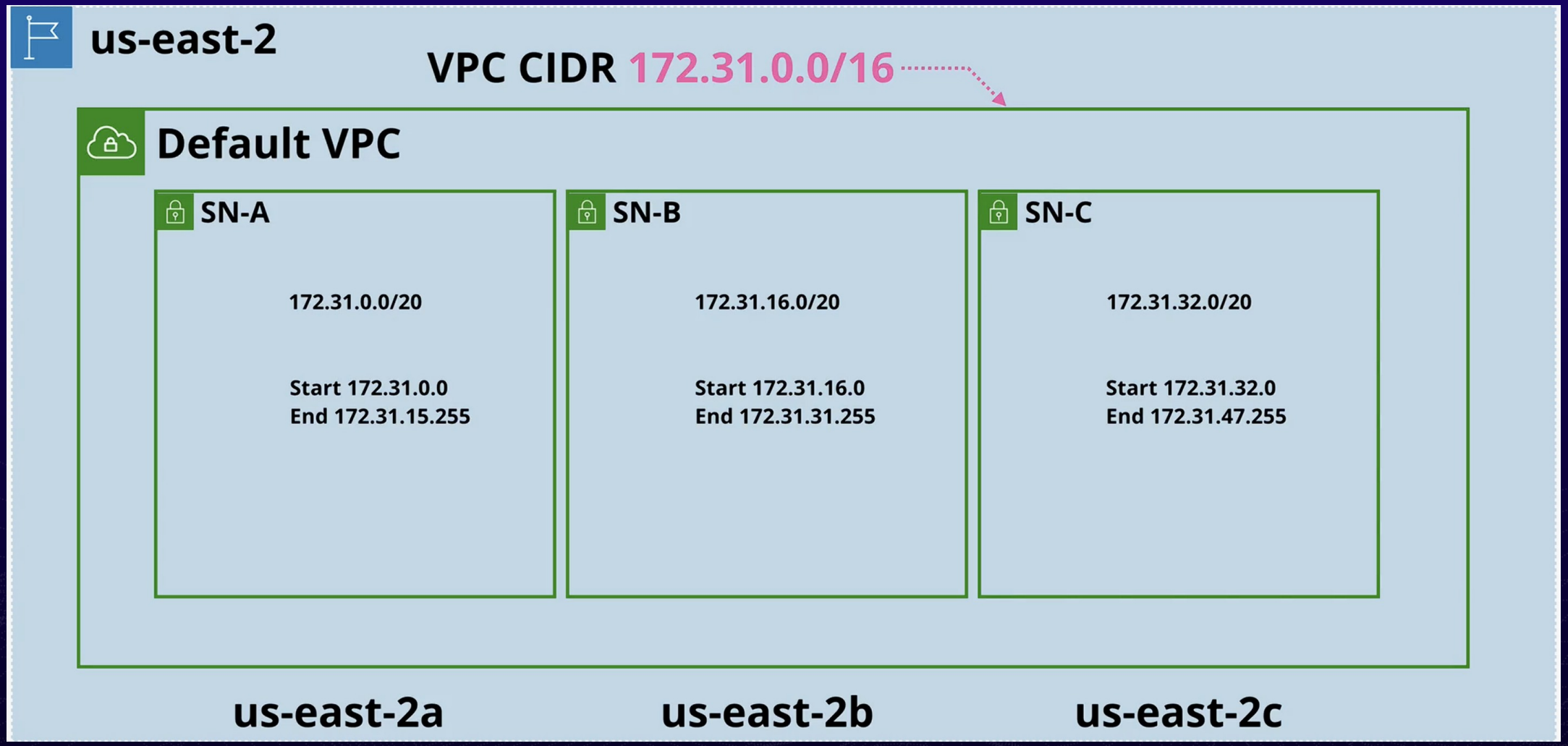
AWS Key Services

Virtual Private Cloud (VPC)

- A VPC = (Can’t communicate)A Virtual Network inside AWS
- A VPC is within 1 account & 1 region
- Private and Isolated unless you decide otherwise
- Two types - Default VPC and Custom VPCs
- **CIDR Block**: Defines the range of IP addresses you can use within the VPC.
- **Subnets**: (Can communicate) Divide your VPC into smaller sections:
  - **Public Subnet**: Directly accessible from the internet (e.g., for web servers).
  - **Private Subnet**: Hidden from the internet (e.g., for databases).
- **Internet Gateway**: Allows resources in a public subnet to access the internet.
- **NAT Gateway**: Allows resources in a private subnet to access the internet securely.

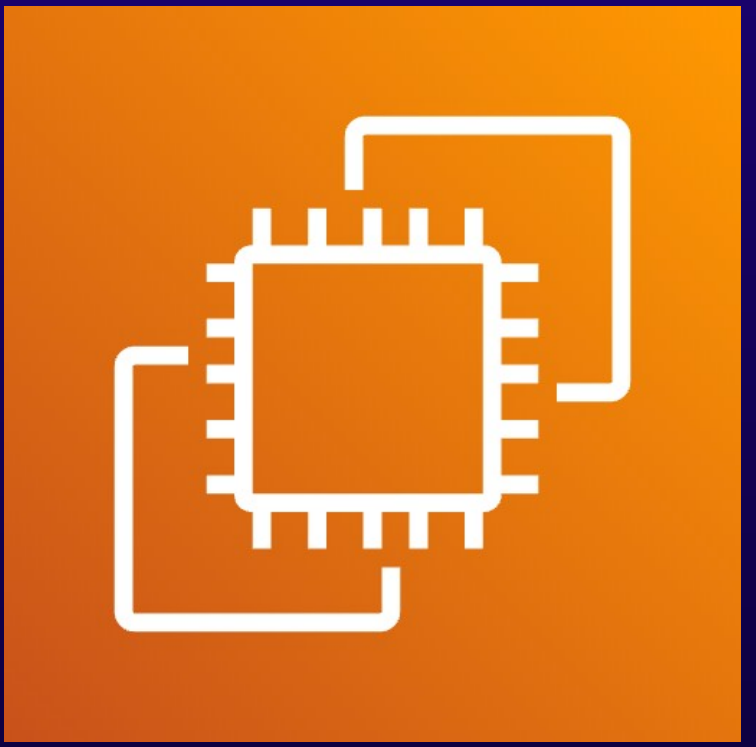
AWS Key Services

Virtual Private Cloud (VPC)



AWS Key Services

Elastic Compute Cloud (EC2)



- **Purpose**: Scalable virtual servers for running applications.
- **Instances**: Virtual machines with configurable CPU, memory, storage, and OS.

- **Key Features**:
  - **Elasticity**: Scale up/down based on demand.
  - **Instance Types**: Choose from a variety of types.
  - **Pricing Models**: On-demand, Reserved.
  - **Storage**: Attach Elastic Block Store (EBS)

- **Networking**: Public/Private subnets in VPC.
- Assign Elastic IP for static public addresses.

- **Best Practices**:
  - Use IAM Roles for access control.
  - Enable security groups for traffic filtering.
  - Automate configuration with user data.

Day 1 Assignment

reate a PDF explaining key AWS services

Instructions:

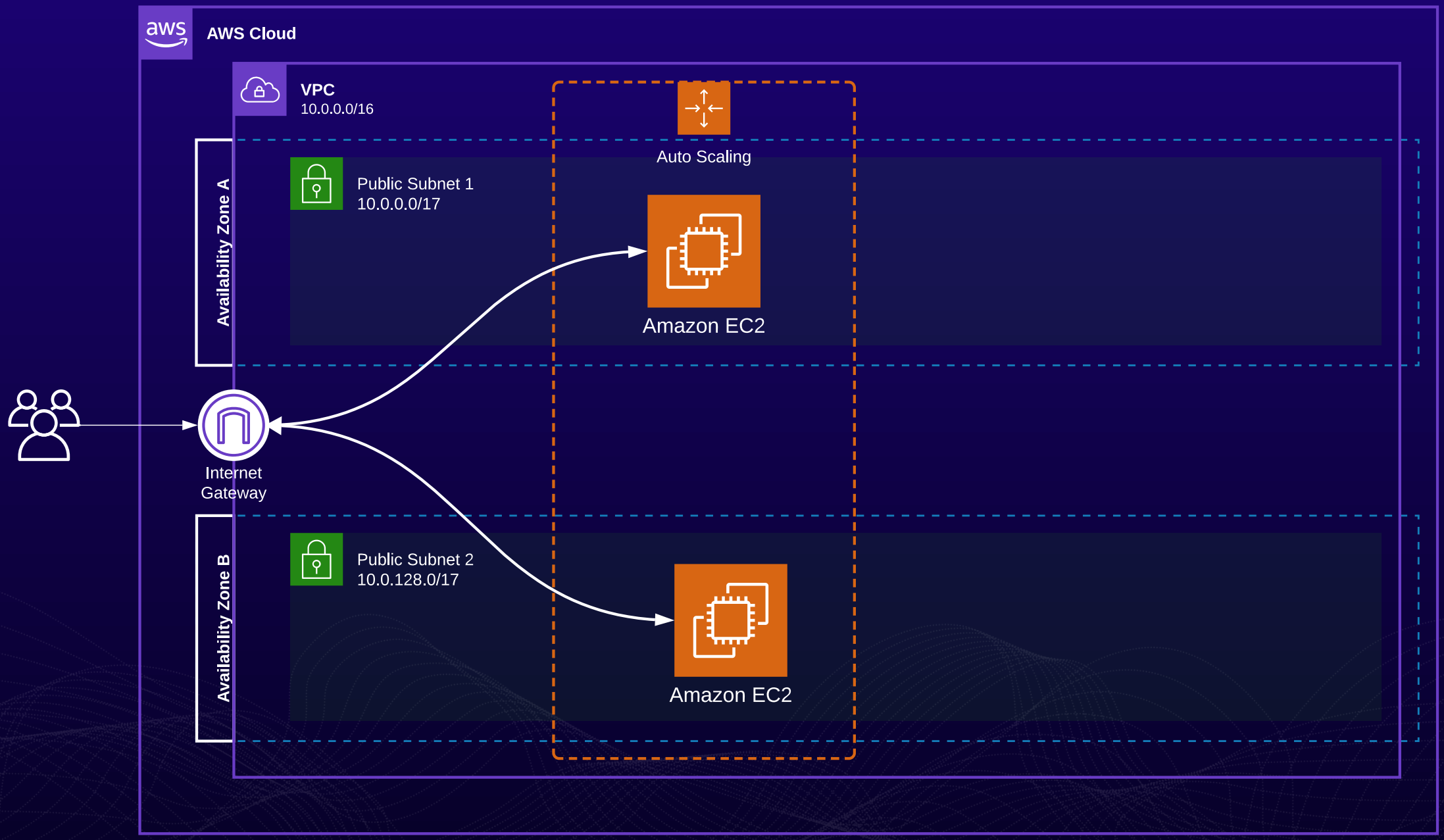
Content: Include the following sections:

1. **IAM (Identity and Access Management)**
2. **VPCs (Virtual Private Cloud)**
3. **EC2 (Elastic Compute Cloud)**
4. **DynamoDB (DDB)**
5. **Lambda**
6. **S3 (Simple Storage Service)**

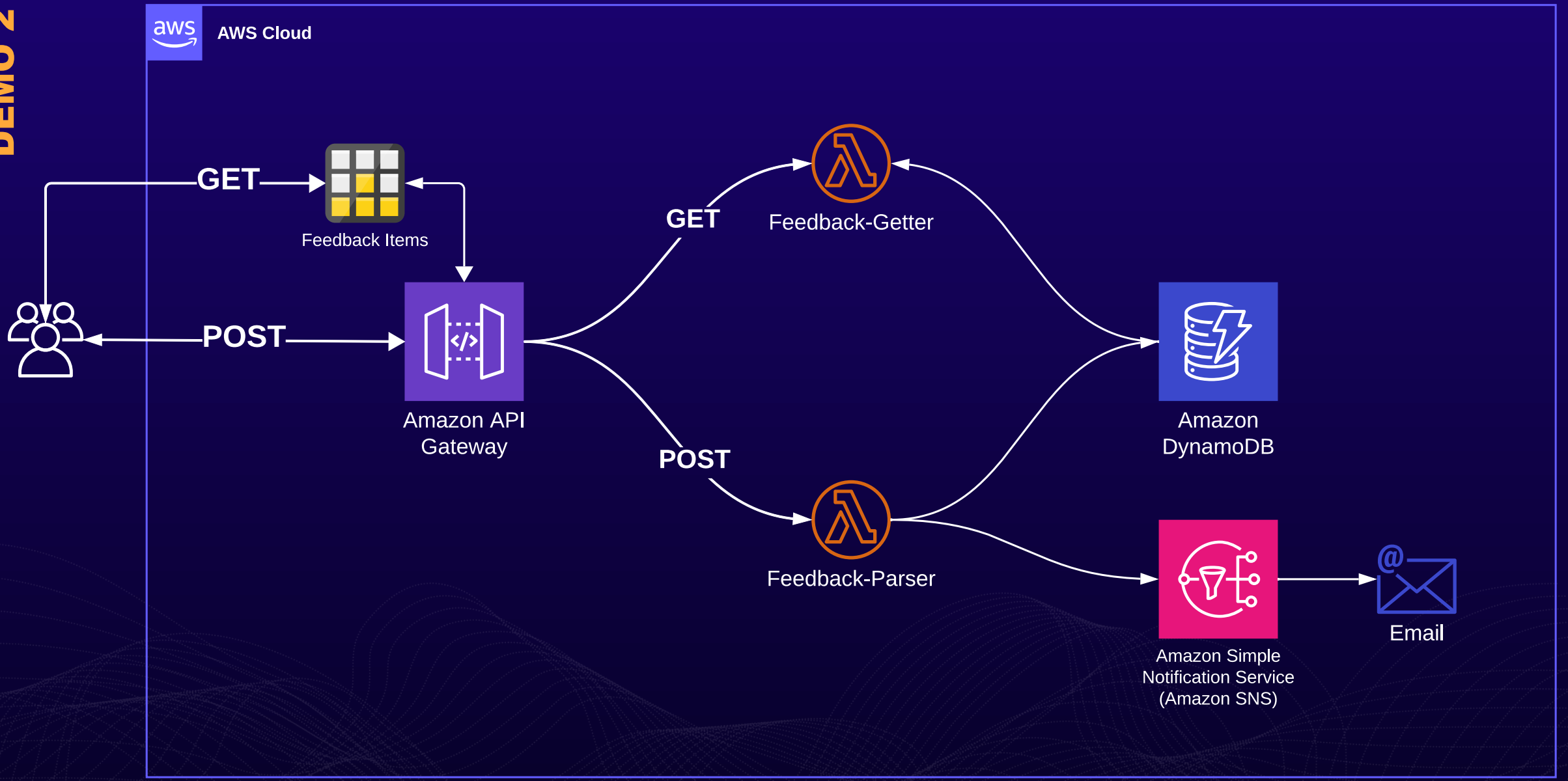
Submission:

- Save the file as *FirstName-LastName-AWS-DayOne.pdf*
- Upload the file to the Drive—create a folder named after you to which only you and I have access
- Deadline Today 13/12/2024 at **11:00 PM**

DEMO 1


aws
AWS Cloud
VPC
10.0.0.0/16
Auto Scaling
Availability Zone A
Public Subnet 1
10.0.0.0/17
Amazon EC2
Internet
Gateway
Availability Zone B
Public Subnet 2
10.0.128.0/17
Amazon EC2

DEMO 2


aws
AWS Cloud
GET
Feedback Items
POST
Amazon API Gateway
GET
Feedback-Getter
POST
Feedback-Parser
Amazon DynamoDB
Amazon Simple Notification Service (Amazon SNS)
Email